PINN_M3DC1

Module 3

Full-Field Dataset Contracts and M3D-C1-Shaped Input/Output

A textbook-style scientific data specification for auditable reduced-MHD surrogate learning

Module purpose

Store accepted numerical fields, parameters, coordinates, provenance, and whole-case splits in explicit versioned schemas, then construct lossless full-field operator views for Modules 4 and 5 without changing the physical meaning fixed by Modules 1 and 2.

Version 1.1 | 11 August 2026

Physics and full-field data-contract note for the synthetic reduced-MHD proof of concept

Preface: why a data contract is part of the physics

Module 2 produces verified numerical trajectories and freezes the revised downstream map

$$(\psi_0(x,y), U_0(x,y), \eta, \nu, t) \longmapsto (\psi(x,y,t), U(x,y,t)) .$$

Module 3 determines whether those trajectories remain scientifically intelligible after they are written to disk, combined into a dataset, split for learning, transformed for optimization, packed into full-field tensors, read by another programming language, and eventually compared with genuine M3D-C1 exports. The learning object is therefore no longer one coordinate row. It is a complete two-dimensional numerical-field input paired with a complete two-dimensional numerical-field target at a declared time.

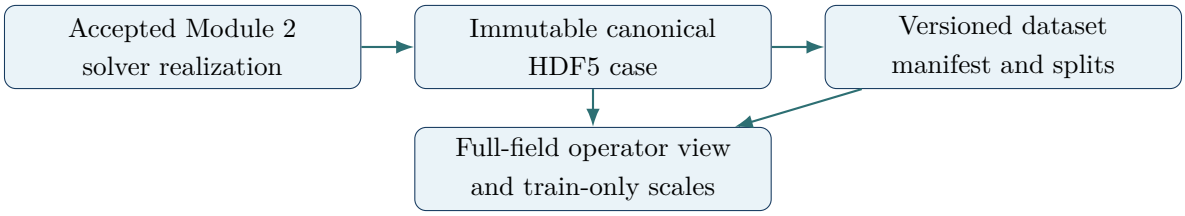
A file format alone cannot answer these questions. An array of numbers does not identify which axis is time, whether current means $J = -\nabla_\perp^2 \psi$ or $J = +\nabla_\perp^2 \psi$, whether the rightmost spatial endpoint is duplicated, whether a field is dimensional or nondimensional, whether a case passed convergence checks, or whether preprocessing used the test set. Those facts determine the scientific meaning of every number. The **data contract** is the set of rules that makes that meaning explicit and machine-checkable.

The initials **I/O** mean *input/output*: how information enters and leaves a computational component. **HDF5** means Hierarchical Data Format version 5, a format that organizes multidimensional datasets, groups, and attributes. **Schema** means a versioned specification of required names, types, shapes, meanings, and relationships. **Provenance** means the recorded history of how an artifact was produced. A **full-field operator example** contains complete numerical arrays rather than isolated scalar samples. **Normalization** here can mean either physical nondimensionalization or a later numerical transform for neural-network training; these are different operations and must remain distinguishable.

Important distinction

"M3D-C1-shaped" describes the architecture of the workflow: coordinates or mesh information, parameters, time-dependent numerical fields, metadata, diagnostics, and an adapter boundary. The revised full-field tensors make this interface more field-oriented, but they do not make the synthetic Cartesian variables numerically equivalent to M3D-C1 variables, make the geometries agree, or validate an M3D-C1 surrogate.

The main dependency chain



The reference case is never overwritten by a preprocessing step. A learning-ready representation is a derived artifact whose parent, transforms, split manifest, and checksums are recorded.

Learning objectives

After working through this module, the reader should be able to:

1. explain why array names, axes, and grid registration are physical statements rather than clerical details;
2. distinguish a physical case, numerical realization, snapshot, grid point, and full-field operator example;
3. state the exact canonical shapes and meanings of ψ, U, J, and ω;
4. explain why accepted reference fields remain unstandardized, unrendered, and immutable;
5. design an HDF5 hierarchy that preserves coordinates, fields, parameters, diagnostics, verification, and provenance;
6. assign stable physical-case, realization, initial-state, and operator-example identities;
7. construct whole-trajectory training, validation, interpolation-test, and extrapolation-test partitions before extracting target times;
8. identify leakage caused by time-snapshot splitting, duplicate realizations, or test-fitted preprocessing;
9. construct $(\psi_0, U_0, \eta, \nu, t) \mapsto (\psi_t, U_t)$ from raw numerical arrays without rendering images;
10. pack fields and scalar context into reversible tensor channels while preserving the parent HDF5 axes and values;
11. explain why A_0 remains case metadata even though the complete U_0 field already encodes it;
12. distinguish the number of physical trajectories from the much larger number of time-conditioned operator pairs;

13. define common-grid and resolution rules appropriate to a full-field neural operator;
14. fit and store invertible field and scalar transforms using training cases only;
15. preserve the spatial-grid metadata needed for spectral physics residuals in Module 5;
16. verify Python–MATLAB field-array and tensor-packing parity;
17. describe the explicit translation work required for genuine M3D-C1 data; and
18. state precisely which claims a Module 3 operator dataset can and cannot support.

Contents

1 Scientific role and acceptance boundary

1.1 The object produced by Module 3

For a physical parameter vector

$$\boldsymbol{\mu}_c = (\eta_c, \nu_c, A_{0,c}), \tag{1.1}$$

Module 2 approximates one accepted reduced-MHD trajectory. Module 3 creates an auditable release

$$\mathcal{D} = (\mathcal{C}, \mathcal{M}, \mathcal{S}, \mathcal{V}, \mathcal{T}), \tag{1.2}$$

where

- $\mathcal{C}$ is a set of accepted canonical case files;
- $\mathcal{M}$ is a manifest describing and indexing those files;
- $\mathcal{S}$ is an immutable whole-physical-case split assignment;
- $\mathcal{V}$ is a declared full-field operator view specifying how initial fields and target times become input-target tensors; and
- $\mathcal{T}$ is the set of declared, invertible preprocessing transforms fitted only from training cases.

For each accepted case and saved target time t_n, the primary version-1 operator example is

$$X_{c,n} = \left(\psi_0^{(c)}(\cdot,\cdot), U_0^{(c)}(\cdot,\cdot), \eta_c, \nu_c, t_n\right), \qquad Y_{c,n} = \left(\psi_n^{(c)}(\cdot,\cdot), U_n^{(c)}(\cdot,\cdot)\right). \tag{1.3}$$

The dots mean the complete two-dimensional numerical arrays on the accepted grid. They do not mean images, contour plots, or isolated coordinate samples.

The manifest and operator-view specification are part of the scientific artifact. A directory of HDF5 files without an authoritative index does not define which files belong to a release, which are accepted, which target times form learning examples, or how they are partitioned.

1.2 Four layers that must not be confused

Layer	Object	Main question
Physical model	Continuous fields and equations	What do $\psi, U, J, \omega, \eta, \nu, A_0$ mean?
Numerical realization	Finite arrays from one solver run	Which grid, time step, integrator, precision, and verification status produced them?

Layer	Object	Main question
Serialized reference	HDF5 datasets plus metadata	Can another reader reconstruct the same logical arrays and meanings?
Learning view	Whole-field input-target tensors derived from accepted trajectories	Which split, target-time rule, channel layout, grid class, scales, and inverse transforms define the neural task?

A neural preprocessing choice must not be written back into the physical-model layer. Similarly, changing a field sign cannot be treated as a harmless file-format revision.

1.3 Acceptance inherited from Module 2

Module 3 ingests only cases for which the Module 2 acceptance record is complete and true. At minimum, the case must have passed:

- operator and sign tests;
- analytic or manufactured-solution checks;
- temporal and spatial convergence checks;
- spectral and current-sheet resolution checks;
- energy, periodicity, gauge, and topology checks;
- finite-value and complete-horizon checks; and
- Python–MATLAB parity at the declared level when parity is required for that release.

Validation checkpoint

The Module 3 validator must fail closed. Missing acceptance metadata means "not eligible," not "probably accepted." A rejected or incomplete case may be retained for debugging, but it cannot appear in an accepted release manifest.

1.4 What Module 3 does not establish

Module 3 does not prove that the solver equations are physically adequate for a stellarator, that a neural network has learned them, or that an M3D-C1 export can be interpreted through the same variables. It establishes a trustworthy interface for the stated synthetic model and a place where a future adapter can make every required translation explicit.

2 Frozen physical semantics

2.1 Domain, coordinates, and endpoint convention

The version-1 domain is

$$\Omega = [0, L_x) \times [0, L_y), \qquad L_x = L_y = 2\pi. \tag{2.1}$$

The spatial grids are

$$x_i = i\Delta x, \qquad \Delta x = L_x/N_x, \qquad i = 0, \ldots, N_x - 1, \tag{2.2}$$

$$y_j = j\Delta y, \qquad \Delta y = L_y/N_y, \qquad j = 0, \ldots, N_y - 1. \tag{2.3}$$

The right endpoints L_x and L_y are not duplicated because they represent the same periodic points as zero. Saved times are an explicit array

$$t_n, \qquad n = 0, \ldots, N_t - 1, \tag{2.4}$$

and need not be reconstructed from a presumed constant save interval.

2.2 Fields and signs

The physical meanings frozen in Module 1 are

$$\boldsymbol{B}_\perp = \nabla\psi \times \hat{\boldsymbol{z}} = (\psi_y, -\psi_x), \tag{2.5}$$

$$\boldsymbol{v}_\perp = \hat{\boldsymbol{z}} \times \nabla U = (-U_y, U_x), \tag{2.6}$$

$$J = -\nabla_\perp^2 \psi, \tag{2.7}$$

$$\omega = +\nabla_\perp^2 U. \tag{2.8}$$

The evolution equations are

$$\partial_t \psi + [U, \psi] = \eta \nabla_\perp^2 \psi, \tag{2.9}$$

$$\partial_t \omega + [U, \omega] = [J, \psi] + \nu \nabla_\perp^2 \omega, \tag{2.10}$$

with bracket

$$[f, g] = f_x g_y - f_y g_x. \tag{2.11}$$

The canonical stored field names are case sensitive:

$$\texttt{psi}, \qquad \texttt{U}, \qquad \texttt{J}, \qquad \texttt{omega}. \tag{2.12}$$

Aliases such as `u`, `phi`, `current`, or `w` are not accepted at the canonical boundary unless an adapter

maps them and records the mapping.

2.3 Primary, prognostic, reconstructed, and derived

The labels "primary" and "derived" depend on purpose.

- The Module 2 time integrator advances ψ and ω.
- It reconstructs U from $\nabla_\perp^2 U = \omega$ with zero spatial mean.
- It derives $J = -\nabla_\perp^2 \psi$.
- The Module 4 and Module 5 neural networks use (ψ, U) as primary targets and derive (J, ω) from spatial derivatives of their predictions.

All four reference fields are stored because they support round-trip verification, derivative-sensitive scoring, and auditing. Metadata must still record the dependency identities in Equation (2.8); four stored arrays are not four independent physical degrees of freedom.

2.4 Gauge conditions and means

The stream function has an arbitrary spatial constant because only its derivatives enter velocity. The frozen gauge is

$$\langle U \rangle (t) = \frac{1}{L_x L_y} \int_\Omega U \, \mathrm{d}A = 0 \quad \text{for every saved time.} \tag{2.13}$$

The spatial mean of ψ is recorded and must remain consistent within a case. Validators check the numerical means rather than trusting a metadata declaration.

2.5 Physical nondimensionalization

All version-1 case values are dimensionless under `dimensionless_v1`. The metadata nevertheless stores the normalization name and reference scales. A value of unity is still a scale choice; omitting it makes a later dimensional reconstruction impossible to audit.

Important distinction : stored truth versus learning values

The canonical fields are stored in the physical nondimensional variables above. They are not mean-centered, standardized, logarithmically transformed, clipped, or cast to single precision for neural training. Those operations belong to a derived learning view.

3 Identity and dataset hierarchy

3.1 Case, realization, snapshot, point, and operator example

Definition

A **physical case** is an initial-value problem with fixed equations, domain, boundary conditions, complete initial fields, physical parameters, and intended time horizon. A **numerical realization** is one discretized solution of that physical case with a specified solver, grid, time step, precision, and source revision. An **operator example** is one complete initial state plus scalar context paired with one complete target state.

One physical case can have several realizations: coarse and fine grids, Python and MATLAB implementations, or repeated verification runs. A snapshot is one full spatial state at t_n. A grid point is one pair (x_i, y_j) inside a snapshot, but it is not the version-1 supervised unit. A single accepted trajectory can yield several examples $(X_{c,n}, Y_{c,n})$ at different target times. Those examples remain correlated because they share the same initial state and dynamical history.

3.2 Why several identifiers are required

The contract uses:

- `physical_case_id`: invariant across scientifically equivalent numerical realizations;
- `case_id`: unique to one canonical numerical realization;
- `initial_state_id`: fingerprint of the canonical initial ψ_0, U_0 fields together with their grid and physical-definition versions; and
- `operator_example_id`: unique to one derived full-field pair, including its parent case, target time, and operator-view version.

The physical identifier is derived from canonical physics-defining metadata: equations version, conventions version, domain, boundary conditions, initial-field definitions or digests, transport parameters, and intended output horizon. The realization identifier additionally includes discretization, integrator, precision, implementation identity, and source/config fingerprints. The initial-state identifier becomes particularly useful when future campaigns contain several different initial-field families rather than only the current A_0 amplitude family.

Important distinction: duplicate-trajectory leakage

Splitting by `case_id` or by `operator_example_id` can place one target time in training and another target time from the same physical trajectory in test. That is not an unseen-case test. All realizations and all operator examples sharing a `physical_case_id` must remain in one partition.

3.3 Opaque identifiers and readable metadata

Identifiers should be stable and opaque, for example

$$\texttt{syn-p-8f3a10c64e9b}, \qquad \texttt{syn-r-54b3027976ad}. \tag{3.1}$$

Floating-point parameters do not belong only in a filename. Strings such as `eta0.001` can be formatted inconsistently, rounded, or confused with a different unit system. Human-readable parameters remain explicit datasets or manifest fields; the identifier provides stable identity.

3.4 Release hierarchy

The logical hierarchy is

$$\begin{aligned} &\text{campaign} \supset \text{dataset release} \supset \text{physical cases} \supset \text{realizations} \\ &\qquad \supset \text{snapshots} \supset \text{operator examples} \supset \text{grid points}. \end{aligned} \tag{3.2}$$

A campaign records the parameter-study intent. A dataset release freezes a particular accepted set, split manifest, operator-view definition, and schema versions. Operator examples are derived from snapshots but never split independently of their physical case. New accepted cases or a changed operator-view definition require a new release or view version even if the parent HDF5 files remain byte-identical.

4 Canonical HDF5 case schema

4.1 Why HDF5 is selected

HDF5 provides named multidimensional datasets, groups, attributes, typed numeric storage, chunking, and lossless compression. Its data model is suitable for self-describing scientific arrays and is supported by Python and MATLAB [1]. The format does not supply scientific meaning automatically; this module supplies the naming and semantic rules.

4.2 Schema identity

Every canonical case has root attributes

$$\texttt{schema\_name} = \texttt{pinn\_m3dc1.case}, \tag{4.1}$$

$$\texttt{schema\_version} = \texttt{1.0.0}, \tag{4.2}$$

$$\texttt{source\_kind} = \texttt{synthetic\_reduced\_mhd}, \tag{4.3}$$

$$\texttt{axis\_order} = \texttt{t,x,y}, \tag{4.4}$$

$$\texttt{learning\_interface} = \texttt{full\_field\_operator\_v1}. \tag{4.5}$$

The schema name answers "which contract?" and the version answers "which revision?" A generic attribute such as `version=1` is insufficient because solver, equations, schema, and dataset releases evolve independently.

4.3 Required hierarchy

```
/
  coordinates/
    x                          float64 [Nx]
    y                          float64 [Ny]
    t                          float64 [Nt]
  fields/
    psi                        float64 [Nt, Nx, Ny]
    U                          float64 [Nt, Nx, Ny]
    J                          float64 [Nt, Nx, Ny]
    omega                      float64 [Nt, Nx, Ny]
  parameters/
    eta                        float64 scalar
    nu                         float64 scalar
    A0                         float64 scalar
    psi_eq                     float64 scalar
  diagnostics/
    time_series/               float64 [Nt] datasets
    events/                    event value/index/presence datasets
  physics/                     versions, IC, BC, gauge, normalization
  numerics/                    grid, FFT, dealiasing, integrator, precision
  verification/                acceptance flags, tolerances, report links
  provenance/                  source, config, platform, dependencies
```

The tree is normative at the group and dataset level. Implementations may add names only under

a documented extension namespace or in a backward-compatible minor schema revision.

4.4 Dataset or attribute?

Use datasets for arrays, values that must be sliced, and values whose exact numeric dtype matters. Use attributes for small descriptive metadata attached to a group or dataset. Required scientific scalars such as η and ν are datasets rather than relying only on attributes, because scalar datasets have unambiguous paths, dtypes, and cross-language read behavior.

Large JSON strings, logs, and full configuration files should not be copied indiscriminately into attributes. Store compact identifiers and checksums inside the case, and preserve the authoritative text artifact beside the dataset release.

4.5 Coordinates

The coordinate datasets satisfy:

$$\text{shape}(x) = (N_x), \tag{4.6}$$

$$\text{shape}(y) = (N_y), \tag{4.7}$$

$$\text{shape}(t) = (N_t), \tag{4.8}$$

with strictly increasing values. Validators require

$$x_0 = 0, \qquad 0 \le x_i < L_x, \tag{4.9}$$

$$y_0 = 0, \qquad 0 \le y_j < L_y, \tag{4.10}$$

$$t_0 = 0, \qquad t_{n+1} > t_n. \tag{4.11}$$

Uniform spacing is recorded and checked for the version-1 pseudo-spectral source. The explicit arrays remain authoritative even if `dx`, `dy`, and `save_interval` are also stored.

4.6 Canonical field axis order

All scalar fields use the logical order

$$q[n, i, j] = q(t_n, x_i, y_j), \qquad q \in \{\psi, U, J, \omega\}. \tag{4.12}$$

Thus the stored shape is $[N_t, N_x, N_y]$. The axis names are stored with every field, and a reader validates dimensions against the three coordinate lengths.

Validation checkpoint : non-square sentinel

Axis parity cannot be verified reliably on a square grid containing symmetric fields. Every reader/writer test must include a non-square, non-symmetric sentinel array such as $q[n, i, j] = 10^4 n + 10^2 i + j$. A transpose then produces an unmistakable failure.

4.7 Memory layout is not logical axis order

NumPy commonly uses row-major contiguous arrays; MATLAB uses column-major storage. HDF5 defines dataset dimensions independently of either language's in-memory stride order. A language binding may require an explicit permutation or transpose to produce the canonical logical array. The reader contract is Equation (4.12), not "whatever shape the library returned."

4.8 Field-specific attributes

Each field records:

- `long_name`;
- `axis_names = [t,x,y]`;
- `normalization = dimensionless_v1`;
- `units = 1` for dimensionless quantities;
- `definition_version`;
- `role = prognostic`, `reconstructed`, or `derived`; and
- the formula dependency for J and ω.

The string `units=1` means dimensionless. It does not mean unknown units.

4.9 Precision, special values, and endianness

Canonical reference fields, coordinates, parameters, and diagnostics are IEEE 754 binary64 values. All required numeric datasets must be finite. NaN and infinity are rejection conditions, not missing-value markers. Optional absent values use an explicit presence flag and a documented sentinel, never an unexplained NaN.

Writers use a declared standard little-endian float64 file datatype for portability. Readers convert to the native representation while preserving numerical values. Any later float32 learning cache is derived and must quantify its error relative to the float64 parent.

4.10 Chunking and lossless compression

Chunk shape affects performance but not scientific meaning. The default field chunk is

$$(1, \min(N_x, 64), \min(N_y, 64)), \tag{4.13}$$

which supports snapshot and spatial-block access without loading an entire trajectory. Lossless compression and byte shuffling may be used and must be recorded. A compression setting never changes the logical checksum defined for uncompressed values.

4.11 Diagnostics and event records

Time-aligned scalar diagnostics use length N_t. Required initial diagnostics are:

- magnetic energy;
- kinetic energy;
- total energy;
- Ohmic dissipation;
- viscous dissipation;
- maximum absolute current;
- reconnected flux; and
- reconnection rate when defined.

An event record stores at least `present`, `time`, `snapshot_index`, `value`, and `definition_version`. Events include current-sheet onset, first current peak, first reconnection-rate peak, topology merger, and confirmation of a post-peak interval. Interpolated event times must say which interpolation method was used.

5 Parameters, physics metadata, and normalization

5.1 Case parameters

The version-1 physical-case parameter vector is

$$\boldsymbol{\mu} = (\eta, \nu, A_0). \tag{5.1}$$

The equilibrium amplitude Ψ_{eq} is stored even when fixed at unity because it enters the initial-condition definition. Derived dimensionless groups may be stored for convenience,

$$S = 1/\eta, \qquad \mathrm{Re} = 1/\nu, \qquad \mathrm{Pm} = \nu/\eta, \tag{5.2}$$

but the primitive parameter datasets remain authoritative.

5.2 Physics metadata

The `/physics` group records:

- equations name and version;
- sign-convention version;
- dimensionality and coordinate system;
- domain periods and boundary conditions;
- guide-field assumption;
- density and incompressibility assumptions;
- initial-condition name, formula version, and parameters;
- gauge and mean policies;
- physical-normalization name and reference scales; and
- the declared relationship of stored primary and derived fields.

5.3 Two normalization layers

Let q_{dim} be a dimensional physical variable and q_0 a physical reference scale. Physical nondimensionalization gives

$$q = \frac{q_{\mathrm{dim}}}{q_0}. \tag{5.3}$$

A later neural standardization may give

$$\widetilde{q} = \frac{q - m_q}{s_q}. \tag{5.4}$$

The first transformation defines the PDE coefficients and physical interpretation. The second improves optimization conditioning. Equation (5.4) must be invertible:

$$q = m_q + s_q\widetilde{q}, \qquad q_{\mathrm{dim}} = q_0(m_q + s_q\widetilde{q}). \tag{5.5}$$

Important distinction

Calling both operations "normalization" without a namespace is dangerous. The contract uses `physical_normalization` for the reduced-MHD scale system and `preprocessing` for neural transforms.

5.4 Reference scales

The physical-normalization record contains the declared length, magnetic-field, density, velocity, time, flux, stream-function, current, and vorticity scales. If a synthetic study chooses numerical scale values of one, the symbolic definitions and dimensional units are still recorded. This preserves the chain needed for later comparison with dimensional exports.

6 Numerical metadata and verification evidence

6.1 Numerical realization metadata

The `/numerics` group records:

- spatial method and version;
- $N_x, N_y, \Delta x, \Delta y$;
- Fourier wave-number ordering and transform normalization;
- dealiasing rule;
- state variables actually advanced;
- Poisson zero-mode policy;
- time integrator and version;
- nominal and actual time-step policy;
- requested output times and actual saved times;
- floating-point precision; and
- restart or continuation ancestry.

The coordinate arrays and field arrays are authoritative values. Duplicated scalars such as N_x exist to make validation and search convenient; they must agree with array shapes.

6.2 Acceptance record

The `/verification` group stores one machine-readable flag per required test, the tolerance, measured value, pass/fail result, and test-definition version. A single `accepted=true` flag is not sufficient by itself. The aggregate status is recomputed from required component flags.

6.3 Convergence references

A production case can refer to a convergence campaign rather than embedding all refinement arrays. The case records the convergence-report identifier, dataset release, compared resolutions, observed differences, and the criterion that selected this realization. The referenced report is content-addressed and retained with the project artifacts.

6.4 Rejected and quarantined data

The allowed lifecycle states are:

$$\texttt{incomplete},\quad \texttt{candidate},\quad \texttt{accepted},\quad \texttt{rejected},\quad \texttt{quarantined}. \tag{6.1}$$

Only `accepted` can enter a release. `rejected` means a known criterion failed. `quarantined` means integrity or semantic compatibility is uncertain. State changes are recorded in provenance rather than erasing history.

7 Provenance, integrity, and reproducibility

7.1 What provenance must answer

For each realization, another researcher should be able to answer:

1. Which configuration initiated the run?
2. Which source revision and implementation produced it?
3. Which dependency and platform versions were used?
4. Which random inputs or perturbations were present?
5. Which verification results accepted it?
6. Has the file changed since the release was frozen?
7. Is this file a restart, migration, conversion, or derived product?

7.2 Required provenance fields

Store the UTC creation time, producer language, producer package version, source revision, dirty-worktree flag, configuration path and SHA-256 digest, command or entry-point identity, dependency lock digest when available, operating system, architecture, and parent artifact identifiers.

Do not store secrets, access tokens, full environment dumps, or private absolute paths. Reproducibility metadata must be useful without leaking credentials or unrelated user information.

7.3 Configuration preservation

The exact canonical run configuration is retained as a release artifact. The case stores its digest and the subset of values required for self-description. A default applied silently by code is still an input and must appear in the resolved configuration.

7.4 Checksums without self-reference

The dataset manifest stores a SHA-256 checksum of every closed HDF5 file. The checksum is external because a file cannot contain a checksum of its own final bytes without a self-reference problem. The case may also store logical payload digests computed over a declared canonical sequence of dataset paths, dtypes, shapes, and uncompressed values.

JSON manifests use a deterministic canonical serialization before hashing. A canonical JSON representation makes equivalent metadata produce the same byte sequence; RFC 8785 defines one such scheme [3].

7.5 Atomic publication

Cases are written under a temporary name, closed, reopened, schema-validated, physics-validated, and checksummed before atomic publication. The release manifest is written only after every referenced file exists and passes validation. An interrupted writer must not leave a partial file with an accepted production name.

Required result

Reproducibility requires both semantic identity and byte integrity. A matching checksum proves that bytes have not changed; it does not prove that the field sign, axis order, or split design is scientifically correct. Schema and physics validators provide that second layer.

8 Dataset release manifest

8.1 Why a second file is required

A case file describes one realization. A dataset manifest describes relationships across cases: membership, order, split assignment, intended task, checksums, duplicate groups, preprocessing, and release history. These facts cannot be represented reliably by inspecting filenames.

The version-1 manifest is UTF-8 JSON conforming to RFC 8259 [2]. JSON is selected because both Python and MATLAB can read it without depending on a language-specific object format. Numeric field arrays remain in HDF5; JSON is not used for bulk arrays.

8.2 Manifest identity

The top-level required fields are:

- `schema_name = pinn_m3dc1.dataset_manifest`;
- `schema_version = 1.1.0`;
- `dataset_id` and `release_version`;
- `created_utc`;
- `case_schema_requirement`;
- `physics_compatibility`;
- ordered `cases`;
- `split_manifest`;
- `task_specification`;
- `operator_view`;
- `preprocessing`;
- `integrity`; and
- `provenance`.

8.3 Case index entries

Each case entry stores:

- relative file path;
- file SHA-256 and byte size;

- `physical_case_id` and `case_id`;
- source kind and acceptance status;
- parameter vector;
- coordinate and field shapes plus a grid-compatibility signature;
- initial-state identifier and initial-field digests;
- schema and conventions versions;
- split and test-regime labels; and
- any duplicate, refinement-family, or parity-family identifier.

Paths are relative to the manifest directory and use forward slashes. Moving a release directory therefore does not invalidate its internal path references.

8.4 One authoritative split assignment

The split may be represented as explicit lists of physical-case identifiers or as labels on case entries, but the manifest must have one authoritative form. If both are stored for convenience, validation requires exact agreement.

8.5 Release immutability

Once published, a release is immutable. Correcting a case, changing a split, adding a required field, or recomputing preprocessing creates a new release. An older release may be marked superseded, but its files and manifest remain available for reproduction.

This follows the central release principle of semantic versioning: published contents are not silently modified, and compatibility is communicated by major, minor, and patch components [4]. The schema and dataset release have independent versions because a dataset can add cases without changing its case schema.

9 Whole-case split construction

9.1 The scientific unit of generalization

The main operator question is whether a model predicts the evolution of an unseen physical trajectory, not whether it predicts a later snapshot from a trajectory it already saw. Therefore the

split unit is the *physical case before operator-pair extraction*. Define disjoint sets

$$\mathcal{P}_{\mathrm{tr}}, \quad \mathcal{P}_{\mathrm{va}}, \quad \mathcal{P}_{\mathrm{ti}}, \quad \mathcal{P}_{\mathrm{te}}, \tag{9.1}$$

for training, validation, interpolation test, and extrapolation test physical-case identifiers. They are pairwise disjoint and together cover the accepted learning release.

Every numerical realization, every saved target time, and every derived operator example associated with one `physical_case_id` inherits the same partition. The order of operations is

$$\boxed{\text{accept trajectories} \rightarrow \text{group physical cases} \rightarrow \text{freeze splits} \rightarrow \text{extract operator pairs}}. \tag{9.2}$$

9.2 Roles of the partitions

Training cases update model parameters and fit preprocessing. Validation cases select architecture, optimization, target-time sampling, operator hyperparameters, and checkpoints. Interpolation tests estimate performance on held-out physical cases supported by the training design. Extrapolation tests probe cases outside the declared training support. Final test results must not drive redesign.

9.3 Interpolation in scalar parameters and initial fields

For version 1, transport interpolation is assessed in the (η, ν) plane and A_0 remains a useful case label even though its effect is represented through U_0. A held-out scalar parameter vector can be called interpolation only under a declared geometric rule such as membership in the convex hull of training parameter vectors.

Initial-field generalization is a separate axis. The present family

$$\psi_0 = \sin x \sin y, \qquad U_0 = A_0(\cos x - \cos y) \tag{9.3}$$

contains one fixed magnetic shape and one fixed flow shape with varying amplitude. Consequently, the current release cannot claim generalization across arbitrary initial-field shapes. Future campaigns with new initial-state families must label whether a held-out initial state lies within or outside the training family.

9.4 Designed split before random split

A designed split is preferred for a small parameter sweep because it guarantees scientifically interpretable interpolation and extrapolation cases. If a larger future campaign uses random partitioning, randomness is permitted only within a predeclared stratification rule and with a saved seed and algorithm version.

9.5 Duplicate and near-duplicate grouping

Before splitting, group cases that share the same physics-defining parameters and canonical initial fields, the same initial condition under a different numerical resolution, Python/MATLAB parity realizations, restart-derived duplicates, migrated copies, or numerically equivalent initial fields within the declared tolerance. The group, not the file or target time, is assigned to a partition.

9.6 No test information in preprocessing

After the split is frozen, every fitted statistic has the form

$$\widehat{\boldsymbol{\alpha}} = \mathrm{FitTransform}(\mathcal{D}_{\mathrm{tr}}), \tag{9.4}$$

where $\boldsymbol{\alpha}$ contains field means, scales, coefficient transforms, time transforms, clipping policies, or sampling weights. Validation and test cases are transformed using $\widehat{\boldsymbol{\alpha}}$ without refitting.

Important distinction : leakage through convenience

Computing a separate field mean, target scale, or spatial interpolation rule from a held-out trajectory reveals information about that trajectory. The operator view must be executable from parent metadata plus training-fitted transforms alone.

9.7 Split freezing and target-time access

The split manifest is written before model development and before target times are converted into learning pairs. Test fields and metrics must not guide architecture, loss weights, target-time sampling, training duration, or checkpoint selection. If repeated test inspection occurs, the set has become a validation set and a new untouched test release is required.

10 Full-field operator views and preprocessing

10.1 A view, not a replacement

A learning-ready operator view is a reproducible derivation from a canonical release. It records

$$\text{view} = (\text{parent release}, \text{split}, \text{target-time rule}, \text{grid class}, \text{channel layout}, \text{transforms}). \tag{10.1}$$

The canonical HDF5 cases remain unchanged.

10.2 Raw full-field operator map

For case c and target time t_n,

$$\boxed{\left(\psi_0^{(c)}, U_0^{(c)}, \eta_c, \nu_c, t_n\right) \longmapsto \left(\psi_n^{(c)}, U_n^{(c)}\right)}. \tag{10.2}$$

Each field symbol in Eq. (10.2) means the complete $N_x \times N_y$ scalar numerical array on the accepted grid. J and ω remain stored verification and evaluation fields; they are not primary output channels.

Important distinction : numerical fields are not images

No rendering step belongs between HDF5 and the neural operator. Contour plots, field-line plots, PNG files, screenshots, color maps, and rasterized figures are presentation artifacts. The learning tensor must be traceable losslessly to the original scalar arrays.

10.3 Input tensor packing

A convenient channel-last logical representation is

$$\boldsymbol{X}_{c,n} \in \mathbb{R}^{N_x \times N_y \times C_{\text{in}}}, \qquad \boldsymbol{Y}_{c,n} \in \mathbb{R}^{N_x \times N_y \times 2}. \tag{10.3}$$

The two mandatory spatial input channels are ψ_0 and U_0. Scalar context may be supplied separately as

$$\boldsymbol{a}_{c,n} = (\eta_c, \nu_c, t_n) \tag{10.4}$$

or broadcast as constant $N_x \times N_y$ channels. Both layouts represent the same physics and the view version must state which is used. Optional deterministic coordinate channels are architecture-specific additions, not replacements for the authoritative x, y arrays.

10.4 Target tensor packing

The primary target channels are

$$\boldsymbol{Y}_{c,n}[:,:,0] = \psi^{(c)}(t_n), \qquad \boldsymbol{Y}_{c,n}[:,:,1] = U^{(c)}(t_n). \tag{10.5}$$

Channel order is explicit and versioned. Packing is reversible: unpacking must reproduce the parent HDF5 arrays exactly before any optional floating-point cast or preprocessing.

10.5 A_0 as metadata rather than duplicate context

Because $U_0 = A_0(\cos x - \cos y)$, the complete U_0 array already contains the perturbation amplitude. The primary operator context therefore does not duplicate A_0. It remains required metadata and

supports the consistency check

$$\|U_0 - A_0(\cos x - \cos y)\| \le \tau_{\mathrm{IC}}. \tag{10.6}$$

A future arbitrary-initial-field dataset may have no scalar A_0 at all.

10.6 Common-grid requirement

A version-1 full-field tensor batch requires a common logical grid. The primary learning release therefore chooses one accepted production realization per physical case with the same (N_x, N_y, L_x, L_y) and identical coordinate arrays. Higher-resolution verification realizations remain linked provenance rather than additional learning cases. Any future restriction, interpolation, padding, or mesh projection is a separately versioned transform with quantified error.

10.7 Field preprocessing

The canonical fields remain in physical nondimensional variables. A derived view may standardize the two physical channels using training-only statistics,

$$\widetilde{\psi} = \frac{\psi - m_\psi}{s_\psi}, \qquad \widetilde{U} = \frac{U - m_U}{s_U}, \qquad s_\psi, s_U > 0. \tag{10.7}$$

The same ψ transform applies to initial and target ψ, and the same U transform applies to initial and target U, unless a different choice is explicitly versioned. Scientific evaluation first inverts these transforms.

10.8 Time and transport-coefficient transforms

Target time may use a training-defined affine map. If positive η and ν span several orders of magnitude, the view may use $\log_{10}\eta$ and $\log_{10}\nu$ followed by training-only affine scaling. These are view transforms, not changes to the canonical case.

10.9 Statistic weighting policy

One trajectory can produce many target times and each field contains many grid values. The primary policy is case-balanced first, then target-time balanced, then area balanced. Dense output cadence or higher resolution may not dominate merely by contributing more scalars.

10.10 Physics derivative metadata for Module 5

The revised physics-informed operator does not obtain spatial derivatives by differentiating a full-field operator. On the periodic structured grid, Module 5 can evaluate spatial derivatives of predicted fields using the declared Fourier wave-number convention, while the derivative with respect to conditioning time may be obtained through automatic differentiation with respect to scalar t. Module 3 therefore preserves exact x, y arrays, domain periods, FFT wave-number ordering and normalization, periodic endpoint convention, any grid transform, and inverse preprocessing maps.

10.11 Clipping, imputation, and augmentation

The primary view performs no output clipping and no imputation. Clipping a held-out scalar input to the training range changes extrapolation into a boundary prediction. Noise, masking, random crops, synthetic image augmentation, or field perturbations are separately versioned experiments and cannot be inserted into the canonical operator view.

11 Operator-pair extraction, batching, and reconstruction

11.1 From one trajectory to time-conditioned pairs

For accepted case c, the pair builder reads ψ_0, U_0 once and associates them with selected target times,

$$(c, n) \longmapsto (X_{c,n}, Y_{c,n}). \tag{11.1}$$

The $t = 0$ pair is useful as an identity and packing test. Later target times are not independent trajectories.

11.2 Operator-example identity and pair manifest

Each example stores or deterministically reconstructs the parent dataset and case digests, physical-case/realization/initial-state identifiers, target snapshot index and exact t_n, operator-view version, preprocessing digest, grid signature, and `operator_example_id`. The pair manifest contains indices and metadata, not duplicated bulk arrays.

11.3 Batch shapes

For batch size B on the common grid, a channel-last loader may return

$$\boldsymbol{X} \in \mathbb{R}^{B \times N_x \times N_y \times C_{\text{in}}}, \qquad \boldsymbol{Y} \in \mathbb{R}^{B \times N_x \times N_y \times 2}, \tag{11.2}$$

plus case identifiers, target times, and any separate context vector. Framework-specific channel-first permutations are explicit and reversible.

11.4 Four counts that must not be conflated

Report separately the number of accepted physical trajectories, numerical realizations retained for provenance, selected time-conditioned operator examples, and scalar grid values contained in those tensors. A dataset with 27 trajectories and 100 target times per trajectory contains 2700 operator pairs, not 2700 independent physical cases.

11.5 Target-time sampling

Training may use all saved target times or a deterministic subset. Uniform-in-time, event-stratified, and curriculum sampling are different learning distributions. A fair Module 4/5 comparison uses the same labeled target pairs unless the experiment explicitly studies data-budget differences.

11.6 Case-balanced sampling

The recommended sampler first selects physical cases, then target times within those cases. This prevents cases with more stored times from controlling the loss.

11.7 Resolution and ragged cases

Canonical releases may retain verification realizations with different N_x, N_y, N_t, but the primary tensor view does not pad or interpolate them automatically. One declared production-grid realization per physical case contributes to the learning view. Multi-resolution studies require explicit scientific restriction/prolongation or mesh-aware methods.

11.8 Full-field evaluation remains authoritative

Validation and test scoring compares complete predicted ψ, U arrays with complete targets. Derived J, ω, energy, topology, current-sheet, periodicity, and PDE-residual diagnostics are then evaluated. Low tensor MSE alone is not evidence of correct reconnection dynamics.

12 Schema and physics validation

12.1 Validation layers

Layer	Examples	Failure meaning
Structural	Required paths, dtypes, ranks, shapes, strings	File does not implement the schema
Relational	Field dimensions match coordinates; indices are valid	Internally inconsistent serialization
Numerical	Finite values, monotone coordinates, gauge tolerance	Invalid numerical contents
Physical	$J = -\nabla_\perp^2 \psi$, $\omega = \nabla_\perp^2 U$, periodicity, diagnostics	Stored meanings or computations disagree
Provenance	Required versions, config digest, parent links	Reproduction chain is incomplete
Release	Checksums, membership, disjoint splits, no duplicates	Dataset assembly is corrupted or leaky

12.2 Shape and coordinate checks

For every field q,

$$\text{shape}(q) = (\text{len}(t), \text{len}(x), \text{len}(y)). \tag{12.1}$$

The validator checks endpoint exclusion, periodic grid spacing, time monotonicity, declared domain lengths, and agreement between redundant metadata and authoritative arrays.

12.3 Derived-field consistency

On the periodic grid, recompute

$$J_{\text{check}} = -\nabla_{\perp,h}^2 \psi, \qquad \omega_{\text{check}} = \nabla_{\perp,h}^2 U \tag{12.2}$$

with the declared discrete spectral operator. Report both absolute and normalized errors, for example

$$\epsilon_J = \frac{\|J - J_{\text{check}}\|_2}{\max(\|J\|_2, \epsilon_{\text{floor}})}. \tag{12.3}$$

The tolerance is versioned and must reflect roundoff, output precision, and whether stored fields were computed before or after any spectral projection.

12.4 Gauge and mean checks

For every snapshot,

$$|\langle U \rangle_h| \le \tau_U. \tag{12.4}$$

The mean of ψ is compared with its recorded initial value. A case-dependent constant offset cannot be repaired silently during ingestion; repair would create a migrated realization with explicit ancestry.

12.5 Diagnostic recomputation

Recompute energies, maximum current, and selected topology diagnostics from stored fields. Stored histories must agree within declared tolerances. This detects stale diagnostics written from an earlier in-memory state or an inconsistent axis permutation.

12.6 Full-field operator-view checks

For every selected pair, validation requires the input ψ_0, U_0 arrays to equal snapshot zero of the parent case, the target arrays to equal the declared parent snapshot before preprocessing, target time to equal the authoritative t_n, all channels to share the declared grid registration, broadcast scalar channels to equal their metadata values, and pack/unpack to recover parent arrays exactly. No rendering, color conversion, resize, or image codec may appear in the canonical provenance path. At $t = 0$, the target must reproduce the initial fields within storage tolerance.

12.7 Manifest checks

The release validator requires:

- every indexed file exists and matches its checksum;
- every case is accepted and schema-compatible;
- no unindexed file is treated as release data;
- physical-case groups are split exactly once;
- train, validation, and test groups are disjoint;
- preprocessing depends only on training identifiers;
- operator-pair indices lie inside their parent trajectories and inherit the parent split; and
- all parent digests and version constraints resolve.

Validation checkpoint : fail before concatenation

Compatibility is checked case by case before any tensors are packed. Once incompatible arrays have been silently stacked or resampled, their original meanings can be difficult or impossible to recover.

13 Python–MATLAB interoperability

13.1 Scientific parity, not identical internal code

Python and MATLAB readers and writers may use different APIs and internal layouts. They must nevertheless agree on the logical HDF5 paths, axis names, numeric values, string encoding, identifiers, checksums, and validation outcomes.

13.2 Mandatory round-trip matrix

Writer	Reader	Required result
Python	Python	Exact schema and value round trip
Python	MATLAB	Canonical logical arrays after explicit mapping
MATLAB	MATLAB	Exact schema and value round trip
MATLAB	Python	Canonical logical arrays after explicit mapping

Every direction uses a small non-square sentinel case and at least one production-like case.

13.3 Operator tensor packing parity

Python and MATLAB independently construct at least one operator example from the same canonical case and must agree on logical ψ_0, U_0, ψ_t, U_t arrays, target time, context values, and channel ordering before framework-specific permutation. A non-square sentinel grid is mandatory because square tensors can hide an x–y transpose.

13.4 Strings, booleans, and scalars

Cross-language failures often arise from metadata rather than fields. The contract therefore specifies UTF-8 strings, scalar numeric datasets with rank zero, and acceptance flags stored as unsigned 8-bit integers with allowed values 0 and 1. Readers convert these representations to native strings and logical values only after schema validation.

Fixed versus variable-length strings must be tested by both readers. Trailing NUL bytes or padded spaces are not part of an identifier. The canonical comparison uses decoded Unicode text.

13.5 Index bases

Serialized snapshot and grid indices are zero based. MATLAB converts to one-based subscripts at its internal boundary:

$$n_{\mathrm{MATLAB}} = n_{\mathrm{stored}} + 1, \tag{13.1}$$

and similarly for i and j. Writing a MATLAB index without subtracting one creates a valid-looking but shifted target snapshot or operator pair.

13.6 Numeric comparison

Values written and read without arithmetic should agree bitwise after dtype conversion. Independently generated solver results are compared with declared physical tolerances, not required to be bitwise identical. The report must distinguish an I/O round-trip test from a solver-parity test.

14 Meaning of M3D-C1-shaped I/O

14.1 The similarity being claimed

The synthetic workflow and a future M3D-C1 workflow can share an abstract record:

$$\text{source metadata} + \text{coordinates/mesh} + \text{parameters} + \text{time-dependent fields} + \text{diagnostics} + \text{splits}. \tag{14.1}$$

This is the sense in which the I/O is M3D-C1-shaped. It demonstrates a disciplined route by which high-fidelity exports could become learning data.

14.2 What cannot be identified by name alone

Synthetic version 1	Possible M3D-C1 export concept	Required adapter question
Cartesian (x, y) periodic grid	Finite-element mesh in toroidal geometry	Which coordinates, elements, basis functions, periodic identifications, and quadrature define the field?
ψ Cartesian flux function	Flux-like exported potential	Are sign, gauge, dimensional scale, geometric factors, and physical definition equivalent?
U Cartesian stream function	Flow-like reduced potential	Which velocity components and metric factors does it generate?
$J = -\nabla_\perp^2 \psi$	Current component or current density	Which component, operator, units, sign, and basis are used?
$\omega = \nabla_\perp^2 U$	Vorticity-like variable	Is it exported, reconstructible, and defined by the same operator?

Synthetic version 1	Possible M3D-C1 export concept	Required adapter question
η, ν, A_0	Transport, equilibrium, and perturbation inputs	Are coefficients scalar, spatially varying, tensorial, dimensional, or closure-dependent?
Two-field incompressible model	Selected extended/reduced MHD option	Which evolved fields, closures, sources, and equations were active?

M3D-C1 adapter boundary

Matching symbols do not establish matching variables. The adapter must derive or document the map from exported quantities to canonical learning variables, including geometry, units, normalization, sign, gauge, interpolation, and information loss.

14.3 Source-native and canonical layers

Genuine exports first enter an immutable **source-native** layer that preserves the original mesh, field names, units, and metadata. An adapter produces a separate canonical layer. It never overwrites the source. The canonical artifact records:

- source file checksums and export-tool version;
- selected M3D-C1 model option and equations;
- field-by-field mapping formulas;
- coordinate and mesh transformation;
- interpolation or projection operator;
- unit and normalization conversion;
- sign and gauge corrections;
- fields dropped, approximated, or synthesized;
- conservation and reconstruction errors; and
- adapter validation results.

14.4 Structured grid is not a universal canonical target

Forcing finite-element data immediately onto the synthetic $[t, x, y]$ grid may erase mesh geometry, basis order, curved elements, and metric information. The future schema therefore uses a source-family discriminator. `synthetic_reduced_mhd` requires the structured-grid groups defined in

Section 4; `m3dc1_native` will require a separate major source schema with mesh topology and field association. A later learning view may project both onto a common representation only after projection error is measured.

14.5 Conditions for an M3D-C1 surrogate claim

The project may use the phrase "validated M3D-C1 surrogate" only after:

1. genuine exports are available across a declared M3D-C1 configuration domain;
2. active equations, closures, geometry, boundary conditions, mesh, and units are known;
3. the adapter passes field and diagnostic reconstruction tests;
4. train/validation/test splits contain independent M3D-C1 cases;
5. accuracy is evaluated against untouched genuine outputs; and
6. discrepancies between the synthetic reduced model and M3D-C1 are reported rather than hidden.

15 Versioning, compatibility, and migration

15.1 Independent version namespaces

The following versions are stored separately:

- case schema;
- dataset-manifest schema;
- equations and sign conventions;
- physical normalization;
- initial-condition formula;
- diagnostic definitions;
- solver implementation;
- preprocessing view;
- split manifest; and
- dataset release.

Changing one does not imply that all others changed.

15.2 Semantic version policy

For a public schema $X.Y.Z$:

- increment Z for a correction that does not change valid instance interpretation;
- increment Y for backward-compatible optional additions; and
- increment X for incompatible names, required fields, shapes, dtypes, axes, signs, units, or meanings.

A change from $[t, x, y]$ to $[t, y, x]$, from $J = -\nabla_\perp^2 \psi$ to $J = +\nabla_\perp^2 \psi$, or from dimensionless to dimensional values is a major change even if a converter is easy to write.

15.3 Reader behavior

A reader declares a version range it supports. It rejects an unknown major version. It may accept a newer minor version only if required names retain their meanings and unknown optional extensions can be ignored safely. It never guesses a missing version from a filename or date.

15.4 Migration rules

A migration is a deterministic, versioned transformation

$$\mathcal{M}_{a \to b} : D_a \longrightarrow D_b. \tag{15.1}$$

It produces a new file with a new `case_id`, preserves the old file, records parent checksums, writes the migration tool revision, and validates both generic structure and known scientific invariants.

Lossless renaming differs from scientific conversion. If a migration changes precision, interpolates a grid, changes a gauge, or reconstructs a field, the numerical error and information loss are reported.

15.5 Deprecation

Deprecated optional names may be read for a stated transition period but are never written by the newest writer. Required names cannot simply disappear in a minor revision. The migration guide lists replacement paths and any change in meaning.

16 Common failure modes

16.1 A transposed field that still looks plausible

On a square symmetric benchmark, exchanging x and y can preserve the shape and produce visually reasonable contours. Non-square sentinels, asymmetric analytic values, and coordinate-aware diagnostics are necessary.

16.2 Endpoint duplication

Storing both $x = 0$ and $x = L_x$ duplicates a periodic point. It changes quadrature weights, creates repeated training rows, and may produce an apparent seam if the values differ by roundoff or interpolation.

16.3 Silent float32 conversion

Single precision may preserve broad ψ contours while degrading J, ω, high-order PINN residuals, and small balance defects. A float32 cache is an evaluated derivative product, not the reference dataset.

16.4 Stale derived fields

A writer can update ψ but accidentally store J from an earlier state. Recomputing J and ω on read detects this.

16.5 One accepted flag with no evidence

A Boolean cannot explain which tolerances were applied or which test failed after a definition changed. Store the component results and recompute the aggregate.

16.6 Time-snapshot split leakage

Different target times from the same trajectory appear in both train and test. The model has already seen the same initial fields and physical parameters, so the score is not an unseen-case operator test. Split complete physical trajectories before extracting pairs.

16.7 Normalization before splitting

Global means, standard deviations, parameter ranges, or clipping thresholds incorporate held-out cases. The test distribution has influenced the training pipeline even if its rows never enter a gradient batch.

16.8 Filename as metadata

Parsing parameters from filenames loses type, unit, formula version, and canonical precision. Rename operations can silently change apparent meaning. Filenames assist humans; dataset contents and manifests are authoritative.

16.9 Rendered images as operator data

Images discard or quantize field precision, coordinates, gauge, units, sign, and the exact relationship among variables. Color maps also introduce presentation-dependent transformations. Rendered figures are not operator inputs or targets; raw scalar arrays are the learning data.

16.10 Silent grid resampling

Resizing an $N_x \times N_y$ field to satisfy a tensor shape changes the represented numerical function and its derivatives. Restriction, interpolation, padding, or mesh projection must be declared and validated; image-style resizing is never acceptable.

16.11 Schema-valid but physics-invalid

A file can have all required paths and still store J with the wrong sign. Structural validation and physical validation are separate mandatory layers.

16.12 M3D-C1 name matching

Mapping a field named `psi` directly to the synthetic ψ without checking geometry, normalization, gauge, and definition is semantic corruption disguised as convenience.

17 Module 3 computational contract

17.1 Inputs

Module 3 receives:

- closed Module 2 candidate case files containing complete numerical trajectories;
- complete verification records and resolved run configurations;
- equations, conventions, IC, diagnostic, normalization, and operator-interface versions; and
- a version-controlled dataset-release configuration.

17.2 Outputs

It produces:

- immutable accepted canonical HDF5 cases and quarantined cases outside the accepted index;
- a versioned JSON dataset manifest and whole-physical-case split manifest;
- a versioned full-field operator-view specification;
- a deterministic operator-pair manifest containing parent-case and target-time indices;
- training-only preprocessing records and integrity checksums; and
- Python–MATLAB round-trip and tensor-packing reports.

17.3 Recommended components

Component	Responsibility
Case inspector	Read candidate metadata without accepting it implicitly.
Schema validator	Validate names, versions, dtypes, ranks, shapes, and allowed values.
Physics validator	Recompute identities, gauge, periodicity, and diagnostic checks.
Identity builder	Construct physical-case, realization, and initial-state identifiers.
Canonical writer	Publish atomic HDF5 cases in the canonical case schema.
Manifest builder	Index membership, hashes, parameters, grids, schemas, and ancestry.
Duplicate grouper	Keep equivalent physical cases and realizations in one split group.
Split builder	Create designed whole-trajectory partitions and freeze their digest.
Operator-view builder	Freeze input channels, context layout, target channels, grid class, and target-time policy.
Pair-index builder	Create deterministic $(\texttt{case\_id}, n)$ pair records without copying field arrays.
Preprocessor fitter	Fit invertible field and scalar transforms on training cases only.
Tensor packer	Pack/unpack arrays and context with explicit reversible channel order.
Round-trip suite	Test Python/MATLAB HDF5 and operator-tensor parity.

Component	Responsibility
Migration tool	Convert explicit old schemas or views without overwriting parents.

17.4 Required test families

- missing path, wrong dtype, wrong rank, and wrong-axis negative tests;
- non-square sentinel HDF5 and tensor round trips;
- field identity, gauge, periodicity, and diagnostic recomputation;
- checksum corruption detection;
- duplicate physical-case and initial-state grouping;
- split disjointness and proof that all target times inherit the parent split;
- test-leakage guards for preprocessing dependencies;
- operator-pair extraction and $t = 0$ initial-target identity;
- pack/unpack exact recovery of ψ_0, U_0, ψ_t, U_t;
- grid-signature mismatch rejection and no-silent-resampling tests;
- rendered-image provenance rejection for the canonical view;
- schema/view-version acceptance and rejection matrix; and
- migration ancestry and scientific-invariant checks.

17.5 Completion criteria

Module 3 implementation is complete only when at least one accepted Module 2 trajectory is serialized and revalidated; both languages read the same logical case values; the release manifest passes; every physical-case group is assigned exactly once before pair extraction; the operator view and pair manifest reproduce declared tensors; preprocessing proves training-only dependency and inverse recovery; $t = 0$ identity and non-square tensor parity pass; deliberate leakage, grid corruption, and rendered-image substitutions are rejected; and a clean command reproduces the release and operator view.

Important distinction

This physics note freezes the intended contract. The repository remains an implementation scaffold until the writers, validators, manifests, operator-view builders, and tests actually exist and pass.

18 Worked miniature example

18.1 A tiny accepted realization

Suppose $N_t = 3$, $N_x = 4$, $N_y = 5$. Every canonical field has shape $[3, 4, 5]$. Define the asymmetric sentinel

$$q[n, i, j] = 10^4 n + 10^2 i + j. \tag{18.1}$$

Then $q[2, 3, 4] = 20304$, so axis exchange or time reversal is unmistakable.

18.2 Construct one operator pair

For target index $n = 2$,

$$X_{c,2} = (\psi[0, :, :], U[0, :, :], \eta, \nu, t_2), \tag{18.2}$$

$$Y_{c,2} = (\psi[2, :, :], U[2, :, :]). \tag{18.3}$$

If scalar context is broadcast, η, ν, and t_2 become constant 4×5 channels. If stored separately, the field arrays are unchanged. Either representation must unpack to the same parent HDF5 values.

18.3 The $t = 0$ identity test

At target index zero,

$$Y_{c,0} = (\psi[0, :, :], U[0, :, :]) \tag{18.4}$$

must equal the mandatory initial-state channels of $X_{c,0}$ before preprocessing. A mismatch reveals target-index, case, axis, or channel errors before training.

18.4 Physical-case grouping example

Python 128^2 production, MATLAB 128^2 parity, and Python 256^2 refinement realizations with the same complete initial fields and transport parameters have three `case_id` values but one `physical_case_id`. The primary operator view chooses the declared production-grid realization; the others remain verification evidence.

18.5 Training-only scale example

If training gives $m_\psi = 0.02$ and $s_\psi = 0.71$, a held-out entry $\psi = 0.80$ becomes $\widetilde{\psi} \approx 1.099$. Refitting or clipping on the held-out trajectory would change the evaluation question.

18.6 Acceptance logic

If a structurally valid case fails $J = -\nabla^2_{\perp,h}\psi$ by orders of magnitude above tolerance, it is rejected or quarantined. Changing the operator view cannot repair an invalid numerical reference.

19 Summary and bridge to Module 4

Module 3 turns accepted numerical trajectories into a stable full-field scientific interface. Canonical cases preserve raw dimensionless coordinates, float64 fields, parameters, diagnostics, equations, numerical settings, verification evidence, and provenance. Logical axes remain $[t, x, y]$; $J = -\nabla^2_\perp\psi$ and $\omega = \nabla^2_\perp U$ remain checkable identities.

For each accepted physical case and selected target time, Module 3 constructs

$$(\psi_0, U_0, \eta, \nu, t) \longmapsto (\psi_t, U_t) \tag{19.1}$$

using complete $N_x \times N_y$ numerical arrays. Scalar context may be separate or broadcast; the layout is versioned and reversible. Rendered images never enter the canonical learning path.

The manifest freezes release membership, integrity, physical-case grouping, split assignment, operator-view definition, grid compatibility, and preprocessing. All realizations and all target times from one physical case remain in one partition. One trajectory can yield many pairs but still counts as one physical trajectory.

The current initial-state family remains narrow: $\psi_0 = \sin x \sin y$ and $U_0 = A_0(\cos x - \cos y)$. Full-field representation does not create initial-condition diversity absent from the solver campaign. Broader operator-generalization claims require additional independently verified initial-field families.

M3D-C1-shaped remains a limited interface claim: the workflow anticipates field arrays or mesh fields, parameters, time, diagnostics, provenance, and an adapter. Genuine M3D-C1 exports must remain source-native until geometry, units, signs, gauges, interpolation, and projection are validated.

Module 4 receives this frozen release for the data-only full-field operator baseline. Module 5 receives the same physical cases and labeled target pairs and adds reduced-MHD constraints using the preserved grid and transform metadata.

A Normative schema dictionary

Path or attribute	Type	Shape	Meaning
`@schema_name`	UTF-8	scalar	`pinn_m3dc1.case`
`@schema_version`	UTF-8	scalar	Case contract version
`@source_kind`	UTF-8	scalar	`synthetic_reduced_mhd`
`@physical_case_id`	UTF-8	scalar	Physics-equivalence group
`@case_id`	UTF-8	scalar	Unique numerical realization
`@initial_state_id`	UTF-8	scalar	Canonical initial-field fingerprint
`@axis_order`	UTF-8	scalar	Literal `t,x,y`
`@learning_interface`	UTF-8	scalar	`full_field_operator_v1`
`/coordinates/x`	float64	$[N_x]$	Half-open periodic x grid
`/coordinates/y`	float64	$[N_y]$	Half-open periodic y grid
`/coordinates/t`	float64	$[N_t]$	Explicit saved times
`/fields/psi`	float64	$[N_t, N_x, N_y]$	Magnetic-flux function
`/fields/U`	float64	$[N_t, N_x, N_y]$	Zero-mean stream function
`/fields/J`	float64	$[N_t, N_x, N_y]$	$-\nabla_\perp^2 \psi$
`/fields/omega`	float64	$[N_t, N_x, N_y]$	$+\nabla_\perp^2 U$
`/parameters/eta`	float64	scalar	Magnetic diffusivity
`/parameters/nu`	float64	scalar	Kinematic viscosity
`/parameters/A0`	float64	scalar	Initial flow amplitude
`/parameters/psi_eq`	float64	scalar	Equilibrium flux amplitude
`/diagnostics/time_series/*`	float64	$[N_t]$	Time-aligned diagnostic
`/diagnostics/events/*/present`	uint8	scalar	0 or 1
`/diagnostics/events/*/time`	float64	scalar	Event time if present
`/diagnostics/events/*/snapshot_index`	int64	scalar	Zero-based nearest/indexed snapshot
`/physics/*`	mixed	scalar	Equations, IC, BC, gauge, normalization
`/numerics/*`	mixed	scalar	Grid, FFT, integrator, precision
`/verification/*`	mixed	scalar	Criteria, values, tolerances, flags
`/provenance/*`	mixed	scalar	Source, config, platform, ancestry

B Illustrative dataset manifest

The example below is explanatory. The machine implementation is validated against the separately versioned manifest schema.

```
{
  "schema_name": "pinn_m3dc1.dataset_manifest",
  "schema_version": "1.1.0",
  "dataset_id": "island-coalescence-synthetic-v1",
  "release_version": "1.0.0",
  "case_schema_requirement": ">=1.0.0,<2.0.0",
  "physics_compatibility": {
    "equations_version": "reduced_mhd_v1",
    "conventions_version": "conventions_v1",
    "physical_normalization": "dimensionless_v1"
  },
  "cases": [
    {
      "path": "cases/syn-r-54b3027976ad.h5",
      "sha256": "<64 lowercase hexadecimal characters>",
      "physical_case_id": "syn-p-8f3a10c64e9b",
      "case_id": "syn-r-54b3027976ad",
      "initial_state_id": "syn-i-2c91f0ea51b7",
      "parameters": {"eta": 0.001, "nu": 0.001, "A0": 0.1},
      "split": "train",
      "acceptance_status": "accepted"
    }
  ],
  "split_manifest": {
    "unit": "physical_case_id",
    "algorithm": "designed_parameter_split_v1",
    "train": ["syn-p-8f3a10c64e9b"],
    "validation": [],
    "test_interpolation": [],
    "test_extrapolation": []
  },
  "task_specification": {
    "task_kind": "full_field_solution_operator",
    "map": "(psi0,U0,eta,nu,t)->(psi_t,U_t)",
```

```
    "split_unit": "physical_case_id"
  },
  "operator_view": {
    "view_version": "full_field_operator_v1",
    "input_field_channels": ["psi0", "U0"],
    "scalar_context": ["eta", "nu", "t"],
    "target_channels": ["psi", "U"],
    "grid_policy": "common_production_grid",
    "pair_record_path": "views/full_field_operator_v1_pairs.json"
  },
  "preprocessing": {
    "fit_partition": "train",
    "record_path": "preprocessing/full_field_operator_v1.json"
  }
}
```

C Compatibility decision table

Change	Version effect	Reason
Correct prose without changing machine interpretation	Patch	No instance meaning changes
Add an optional diagnostic with a unique path	Minor	Old readers can ignore it safely
Add a new required dataset	Major	Old valid instances no longer satisfy the contract
Rename `psi` to `flux`	Major	Required path changes
Change axis order to $[t, y, x]$	Major	Every field index changes meaning
Change float64 reference fields to float32	Major	Precision contract changes
Change current sign	Major plus conventions version	Physical meaning changes
Change IC formula at the same parameters	IC version and new physical ID	Different initial-value problem
Add accepted cases without changing schema	Dataset release increment	Membership changes, schema does not

Change	Version effect	Reason
Move files while preserving relative manifest structure	New packaged release or identical logical release	Byte/path inventory must remain auditable

D Acceptance checklist

Per-case checks

- ☐ Required schema, source, case, physical-case, and axis attributes exist.
- ☐ Coordinate arrays are float64, finite, monotone, and consistent with the half-open domain.
- ☐ Every field is float64 with logical shape $[N_t, N_x, N_y]$.
- ☐ Parameters are finite and agree with the resolved configuration.
- ☐ $J = -\nabla^2_{\perp,h}\psi$ and $\omega = \nabla^2_{\perp,h}U$ pass declared tolerances.
- ☐ U satisfies its zero-mean gauge at every saved time.
- ☐ Diagnostics recompute within tolerance.
- ☐ All required Module 2 verification components pass.
- ☐ Provenance, source revision, config digest, and dependencies are present.
- ☐ File reopens cleanly after atomic publication.

Release checks

- ☐ Every manifest file exists, matches its checksum, and is accepted.
- ☐ No physical-case identifier appears in more than one partition.
- ☐ Every accepted release member has exactly one split assignment.
- ☐ Interpolation and extrapolation test labels are reproducible.
- ☐ Preprocessing dependencies are a subset of training case identifiers.
- ☐ Operator-pair indices are valid, inherit the parent split, and reproduce identical target times.

☐ $t = 0$ identity and pack/unpack recovery pass; Python and MATLAB readers agree on sentinel, production arrays, and tensor channels.

☐ Dataset, split, operator-view, pair, and preprocessing manifests have independent digests.

☐ Release documentation states the claim boundary and known limitations.

E Symbol table

Symbol	Meaning	Status
x, y	Periodic Cartesian coordinates	dimensionless
t	Saved physical time coordinate	dimensionless
N_x, N_y, N_t	Coordinate and snapshot counts	integers
L_x, L_y	Domain periods	2π baseline
ψ	Magnetic-flux function	stored float64
U	Flow stream function	stored float64
J	Current variable $-\nabla_\perp^2 \psi$	stored/checkable
ω	Vorticity $+\nabla_\perp^2 U$	stored/checkable
η	Magnetic diffusivity	case parameter
ν	Kinematic viscosity	case parameter
A_0	Initial flow perturbation amplitude	case metadata; encoded in U_0
Ψ_{eq}	Equilibrium flux amplitude	stored parameter
$\boldsymbol{\mu}$	Physical-case parameter vector (η, ν, A_0)	solver/case descriptor
$\mathcal{D}$	Dataset release object	cases plus contracts
$\mathcal{P}_{\mathrm{tr}}$	Training physical-case set	split
$\mathcal{P}_{\mathrm{va}}$	Validation physical-case set	split
$\mathcal{P}_{\mathrm{ti}}$	Interpolation-test physical-case set	split
$\mathcal{P}_{\mathrm{te}}$	Extrapolation-test physical-case set	split
m_q, s_q	Training-only location and scale for q	preprocessing
$\widetilde{q}$	Neural transformed value	derived view
$X_{c,n}, Y_{c,n}$	Full-field operator input and target	derived learning pair
τ_U	Gauge tolerance	versioned criterion
τ_{IC}	Initial-field consistency tolerance	versioned criterion

F Acronym and terminology table

Term	Meaning
API	Application programming interface
Canonical	The authoritative representation defined by the current contract
Case	One complete numerical realization unless qualified as physical case
Checksum	Digest used to detect changed bytes or logical payloads
Dataset	A complete release, or an HDF5 multidimensional array when used in HDF5 context
FAIR	Findable, Accessible, Interoperable, and Reusable data principles
Gauge	Convention fixing a physically irrelevant additive freedom
HDF5	Hierarchical Data Format version 5
I/O	Input/output
IC	Initial condition
JSON	JavaScript Object Notation, a language-independent text interchange format
Manifest	Authoritative index of release membership and relationships
M3D-C1	Extended-MHD simulation framework based on high-order C^1 finite elements
MHD	Magnetohydrodynamics
Migration	Versioned conversion from one schema to another
Normalization	Physical nondimensionalization; neural transforms use the separate term preprocessing
Neural operator	Learned map whose input or output is a complete function or discretized field
Operator example	Complete initial-state/context input paired with one complete target state
Operator view	Versioned rule deriving field tensors and context from canonical trajectories
Pair manifest	Parent-case and target-time index for operator examples; bulk arrays stay in HDF5
PINN	Physics-informed neural network
Provenance	Recorded production and ancestry history of an artifact
Realization	One numerical solution of a physical case under specified numerical settings
RMHD	Reduced magnetohydrodynamics; the exact reduction must be stated
Schema	Versioned rules for required names, types, shapes, and meanings
SHA-256	Secure Hash Algorithm with a 256-bit digest, used here for integrity
Split leakage	Held-out information entering training, design, or preprocessing
UTF-8	Unicode text encoding used by manifest and metadata strings
Tensor channel	One semantically named $N_x \times N_y$ numerical array in a packed tensor

Term	Meaning
View	Reproducible derived representation for a learning task

G Concept questions and answer sketches

Questions

Q1. Why is a field name and axis order a physical statement?

Q2. Why are `physical_case_id`, `case_id`, and `initial_state_id` distinct?

Q3. What is one supervised example in the revised Module 3 contract?

Q4. Why can one trajectory produce many operator pairs without becoming many independent cases?

Q5. Why must physical-case splitting occur before target-time pair extraction?

Q6. Why are canonical fields not standardized in place?

Q7. Why are rendered contour images forbidden as canonical neural inputs?

Q8. Why does the operator not need A_0 as a duplicate primary input in version 1?

Q9. Why must A_0 still be stored as metadata?

Q10. What is the version-1 full-field operator map?

Q11. Why is a common production grid useful for the first operator baseline?

Q12. Why is image-style resizing unacceptable for field tensors?

Q13. Why are test-fitted field scales leakage?

Q14. What does the $t = 0$ identity test detect?

Q15. What is the difference between HDF5 round-trip parity and tensor-packing parity?

Q16. Why does full-field input not yet prove generalization to arbitrary initial conditions?

Q17. What does M3D-C1-shaped mean after the redesign?

Q18. What additional evidence is required for an M3D-C1 operator-surrogate claim?

Answer sketches

A1. Names and axes select definitions, signs, coordinates, gauge, normalization, and spatial registration.

A2. They identify a physics-equivalence group, one numerical artifact, and one canonical initial field state.

A3. A complete pair $(\psi_0, U_0, \eta, \nu, t_n) \mapsto (\psi_n, U_n)$ built from whole numerical arrays.

A4. All targets share the same initial state and dynamical history, so they are correlated observations of one trajectory.

A5. Otherwise one time from a trajectory can enter training while another time from the same trajectory appears in test.

A6. Standardization is training-release dependent and reversible; overwriting destroys the accepted reference.

A7. Rendering quantizes or transforms values and introduces presentation-dependent color mappings.

A8. $U_0 = A_0(\cos x - \cos y)$ already encodes the amplitude point by point.

A9. It defines the generating case, supports audits, and enables an initial-condition consistency check.

A10. $(\psi_0, U_0, \eta, \nu, t) \mapsto (\psi_t, U_t)$ on the complete accepted grid.

A11. It enables lossless tensor batching without adding an interpolation problem to the first experiment.

A12. Resizing changes the represented numerical function and its derivatives.

A13. They use held-out target-distribution information to define the training representation.

A14. It catches wrong target indexing, case mismatch, axis transposition, channel swap, and preprocessing errors.

A15. HDF5 parity checks serialization; tensor parity checks the derived representation actually supplied to the learner.

A16. The solver campaign varies mostly the amplitude of one fixed flow shape and uses one fixed magnetic shape.

A17. The workflow is field- and metadata-oriented with an explicit adapter boundary; it still uses synthetic reduced-MHD data.

A18. Genuine independent M3D-C1 exports, a validated adapter, known field definitions and closures, held-out M3D-C1 cases, and renewed operator scoring.

H References and standards

References

[1] M. Folk and E. Pourmal, "Balancing Performance and Preservation: Lessons Learned with HDF5," US DPIF Workshop, NIST, Gaithersburg, Maryland, 2010, https://docs.hdfgroup.org/archive/support/pubs/papers/HDFandpreservation_NIST_2010_paper_Folk.pdf.

[2] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, Internet Standard, December 2017, https://www.rfc-editor.org/info/rfc8259/.

[3] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, June 2020, https://www.rfc-editor.org/info/rfc8785/.

[4] T. Preston-Werner, "Semantic Versioning 2.0.0," https://semver.org/.

[5] M. D. Wilkinson et al., "The FAIR Guiding Principles for scientific data management and stewardship," *Scientific Data*, volume 3, article 160018, 2016, https://doi.org/10.1038/sdata.2016.18.